

RI402
Dizajn kompajlera
dr.sc. Edin Pjanić

Pregled predavanja

- Sintaksna (sintaktička) analiza - parser
 - Terminologija i osnovni principi

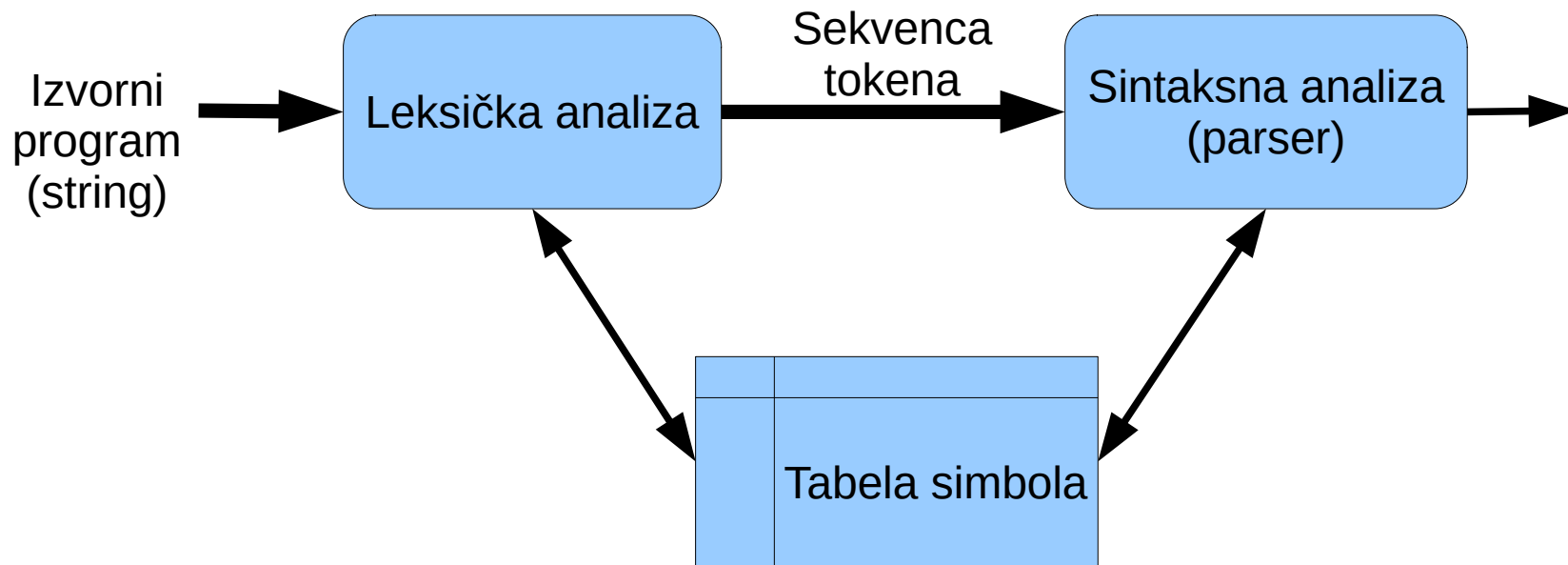
Za čitanje: Dragon Book str. 191

Kompajler

- Leksička analiza (leksiranje)
- **Sintaksna analiza (parsiranje, raščlanjivanje)**
- Semantička analiza
- Generisanje koda

- Leksička analiza:

- Klasifikacija podstringova programa u skladu sa njihovom ulogom.
- Token: <klasa tokena, atributi>



Parser - sintaksna analiza

- Parser od leksera uzima sekvencu tokena
 - slijeva udesno
- Cilj: formirati strukturu programa (stablo parsiranja ili AST)
- Tri tipa parsera:
 - Top-down
 - Bottom-up
 - Univerzalni
- Regularni izrazi nisu dovoljni za ovaj zadatak
 - Hijerarhijska struktura programa
 - Regularni jezici ne mogu “brojati”. Npr. $\{ \binom{n}{n}^n \mid n \geq 0 \}$

Parser

- Primjer:

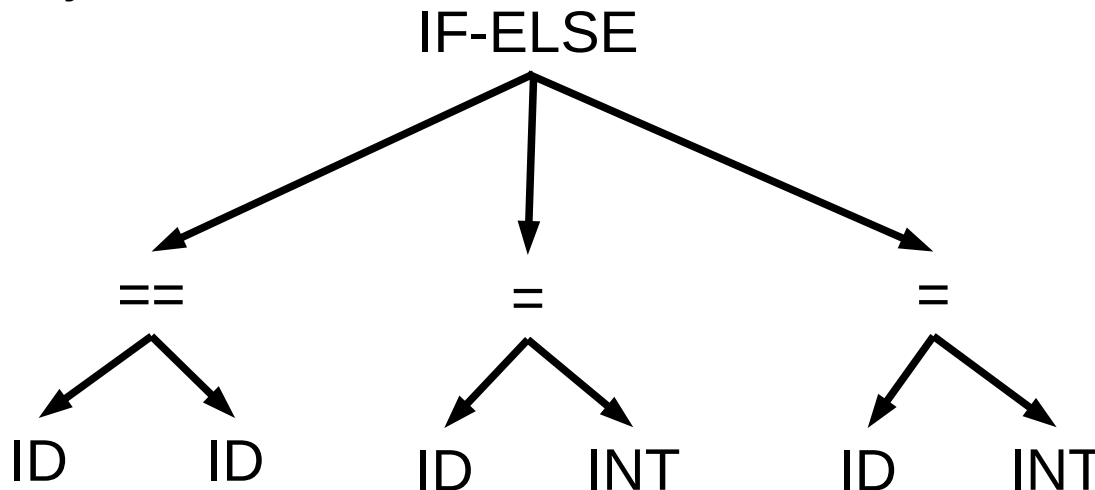
`if (x == y) a = 1; else a = 2;`

- Ulaz u parser, tj. izlaz iz leksera, je sekvenca tokena:

IF (ID == ID) ID = INT ; ELSE ID = INT ;

Možemo zapisati: IF (ID == ID) ID = INT ; ELSE ID = INT ;

- Izlaz iz parsera je stablo:



- Potrebno je opisati i prepoznati neispravnu sekvenca tokena.

Kontekstno-neovisna gramatika

(Context-free grammar - CFG)

- Programi imaju rekurzivnu strukturu.
- Npr. U jezicima tipa C/C++ neki (bilo koji) izraz, označimo ga sa `EXPR`, može biti na sljedećim mjestima :

```
if ( EXPR ) EXPR; else EXPR;
```

```
while (EXPR) EXPR;
```

```
EXPR;
```

```
...
```

- Kontekstno-neovisna gramatika (kontekstno-slobodna):
 - Način zapisivanja rekurzivne strukture programa (npr. Backus-Naur form – BNF notacija).
 - Definisana pravila za neki izraz važe neovisno o kontekstu.

Kontekstno-neovisna gramatika

Opisuje kako treba kombinovati simbole da bi se proizvela sintaksno ispravna sekvenca simbola.

- Sastoji se od:
 - Skupa terminalnih simbola - *terminala* T (klase tokena)
 - Skupa neterminalnih simbola - *neterminala* N (sintaktičke varijable)
 - Početnog simbola S , $S \in N$
 - Skupa *produkcijskih pravila* (produkcija)

$$X \rightarrow Y_1 Y_2 \dots Y_n$$

gdje su:

$$X \in N$$

$$Y_i \in N \cup T \cup \{ \varepsilon \}$$

- Neka je $L(G)$ skup svih stringova koji se mogu generisati iz gramatike G .
- Ako je G kontekstno-neovisna gramatika onda je $L(G)$ kontekstno-neovisan jezik.

Produksijska pravila

Definišu (opisuju) načine na koji se terminalni i neterminalni simboli mogu kombinovati kako bi formirali stringove datog jezika.

$$X \rightarrow Y_1 Y_2 \dots Y_n$$

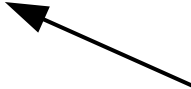
ili

$$X ::= Y_1 Y_2 \dots Y_n$$



Lijeva strana:
glava produkcijskog pravila (head)

- Neterminalni simbol

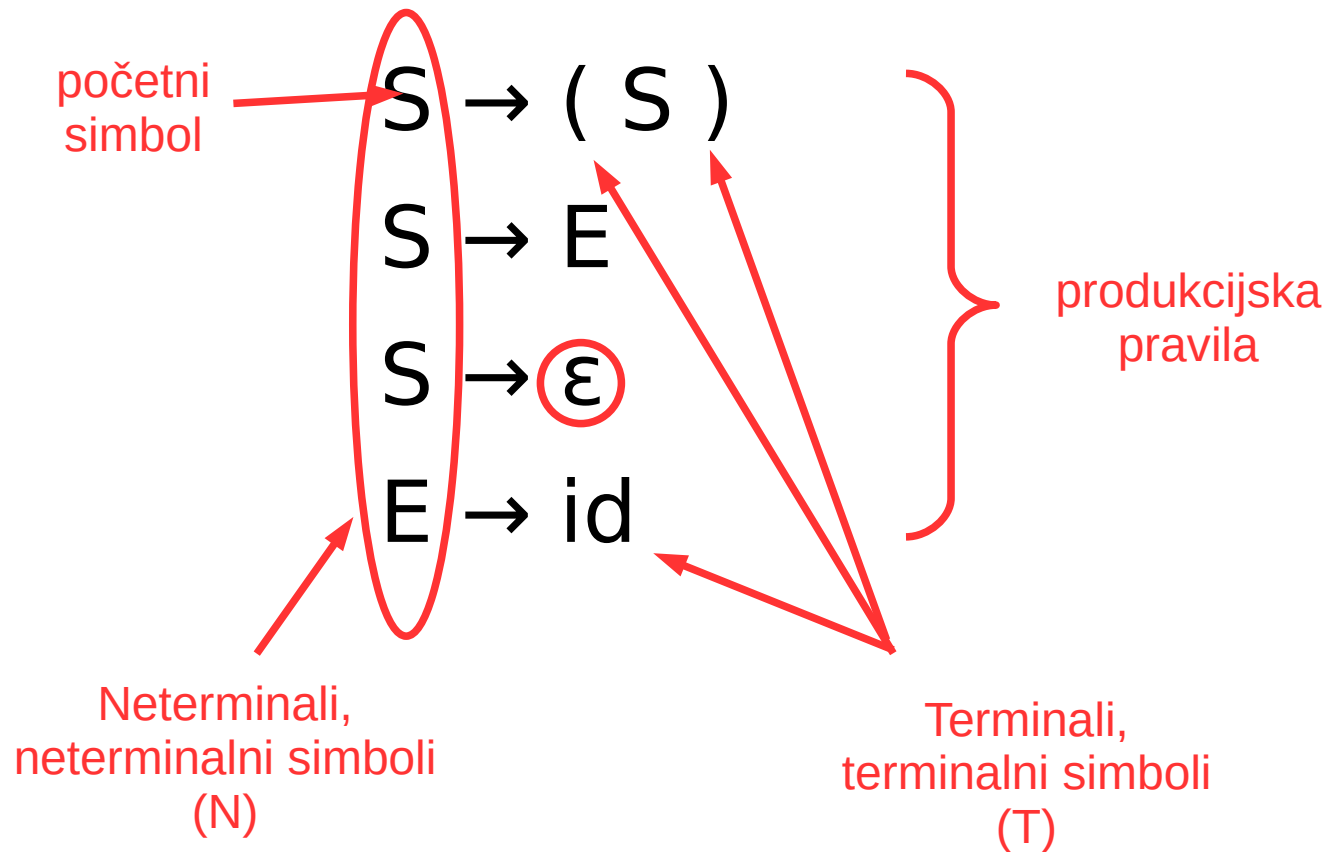


Desna strana:
tijelo produkcijskog pravila (body)

- Terminalni simboli
- Neterminalni simboli

Tijelo produkcijskog pravila definiše jedan način na koji se terminalni i neterminalni simboli mogu kombinovati kako bi proizveli validan string kojeg predstavlja glava pravila.

Primjer gramatike (CFG)



$N = \{ S, E \}$

$T = \{ (,), id \}$

$S \rightarrow \text{lijzagr } S \text{ deszagr}$

$S \rightarrow E$

$S \rightarrow \epsilon$

$E \rightarrow id$

Terminali (terminalni simboli) su elementi za koje nema definisanih produkcijskih pravila, obično tokeni.

Ako imamo više produkcijskih pravila za isti simbol, sva takva pravila imaju isti prioritet, odnosno ravnopravna su (pravilo komutativnosti).

Derivacija – izvođenje stringova jezika

- Validni stringovi jezika definisanog gramatikom se mogu izvesti primjenom produkcijskih pravila.
- Produkcijska pravila se primjenjuju na sljedeći način:
 - kreće se od početnog simbola (neterminalni)
 - neterminalni simbol se zamjenjuje tijelom jednog produkcijskog pravila za taj neterminalni simbol, npr.
 $X \rightarrow Y_1 Y_2 \dots Y_n$
 - prethodni korak se ponavlja sve dok ima neterminalnih simbola u izrazu (produkcijskom pravilu)
 - postupak se prekida kad se dobije izraz u kojem su samo terminalni simboli, odnosno traženi string.
- **Derivacija:** postupak izvođenja jednog stringa krećući od početnog simbola
- Ovaj algoritam opisuje i način generisanja stabla parsiranja. (...)

Derivacija

- Svaka primjena produkcijskog pravila je korak u *derivaciji*.
- Ako gramatika sadrži produkcijsko pravilo:
 - $X \rightarrow Y$ Možemo čitati kao “iz **X** (u jednom koraku) proizilazi **Y**”
- Derivacijski korak koji koristi to pravilo se zapisuje kao:
 - $\alpha X \beta \rightarrow \alpha Y \beta$, gdje su α i β izrazi sa neterminalima i terminalima

- Ako imamo sekvencu koraka u derivaciji:

$$S \rightarrow (S) \rightarrow (E) \rightarrow (id)$$

onda je možemo zapisati kao:

$$S \rightarrow^* (id)$$

Možemo čitati kao “iz **S** nakon nula ili više koraka proizilazi **(id)**”

- Derivaciju

$$S \rightarrow^* (id)$$

Možemo formalno zapisati:

$$\begin{aligned} \alpha \rightarrow^* \beta &\iff \alpha = \beta \text{ ili} \\ &\alpha \rightarrow \beta \text{ ili} \\ &\text{Postoji } \gamma, \alpha \rightarrow \gamma \text{ i } \gamma \rightarrow^* \beta \end{aligned}$$

Takođe: ako je $\alpha \rightarrow \beta$ i $\beta \rightarrow \gamma$, onda $\Rightarrow \alpha \rightarrow \gamma$

- Ako je $S \rightarrow^* \alpha$, gdje je S početni simbol gramatike G:
 - Kažemo da je α **rečenični oblik** od G
- **Rečenica** gramatike G je rečenični oblik koji se sastoji samo od terminalnih simbola.
- **Jezik** gramatike G je skup njenih rečenica.

Derivacija

- Dakle, derivacija je sekvenca koraka koja:
 - Počinje početnim simbolom
 - U svakom koraku derivacije aplicira se jedno produkcijsko pravilo.
 - Završava stringom koji se sastoji samo od terminalnih simbola

$$S \rightarrow^* x_1 x_2 \dots x_n$$

- Jezik koji se može generisati takvom gramatikom:

$$L(G) = \{ x_1 x_2 \dots x_n \mid x_i \in T, S \rightarrow^* x_1 x_2 \dots x_n \}$$

- Ako dvije gramatike generišu isti jezik kažemo da su ekvivalentne.

Primjer

- Neka je zadana gramatika:

$$S \rightarrow E$$
$$E \rightarrow E + E$$
$$| E * E$$
$$| (E)$$
$$| id$$

Koji su stringovi (rečenic

- Mogući koraci za derivaciju:

$$S \rightarrow^* id + id * id$$
$$S$$
$$S \rightarrow E$$
$$\rightarrow E + E$$
$$\rightarrow id + E$$
$$\rightarrow id + E * E$$
$$\rightarrow id + id * E$$
$$\rightarrow id + id * id$$

Derivacijom se provjerava da li neki string pripada datom jeziku.

Ako postoji derivacija za dati string onda string pripada tom jeziku.

Stablo parsiranja

- Stablo parsiranja je grafička predstava derivacije, gdje se vidi i redoslijed primjene produkcijskih pravila.
- Prilikom izgradnje stabla parsiranja, redoslijed dodavanja čvorova je identičan redoslijedu koraka derivacije.
- Listovi stabla, čitani slijeva udesno, predstavljaju ulazni string.

Gramatika:

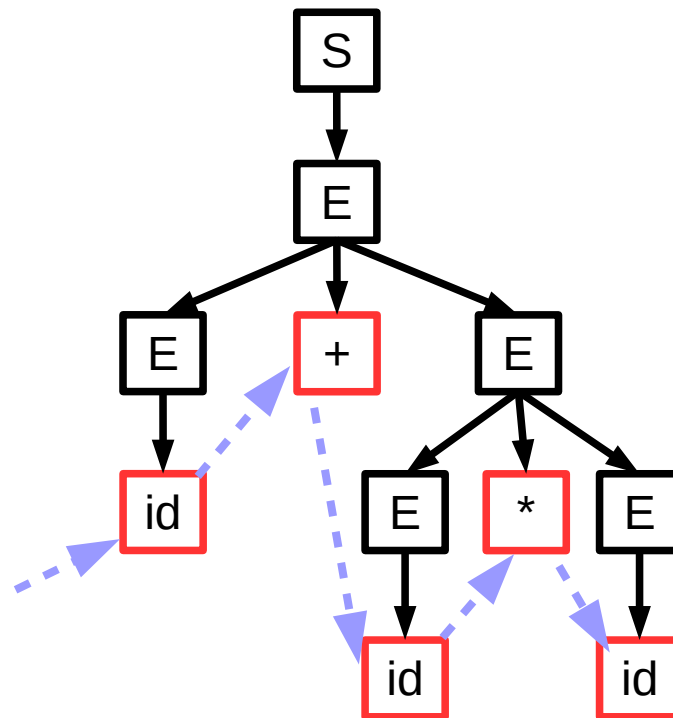
$S \rightarrow E$

$E \rightarrow E + E$

| $E * E$

| (E)

| id



Listovi stabla pri izgradnji stabla sadrže terminale i neterminale.

Čitajući slijeva udesno dobijamo rečenične oblike koje nazivamo **produkcija** ili **margina** stabla parsiranja

$id + id * id$

Primjer 1 – lijeva derivacija

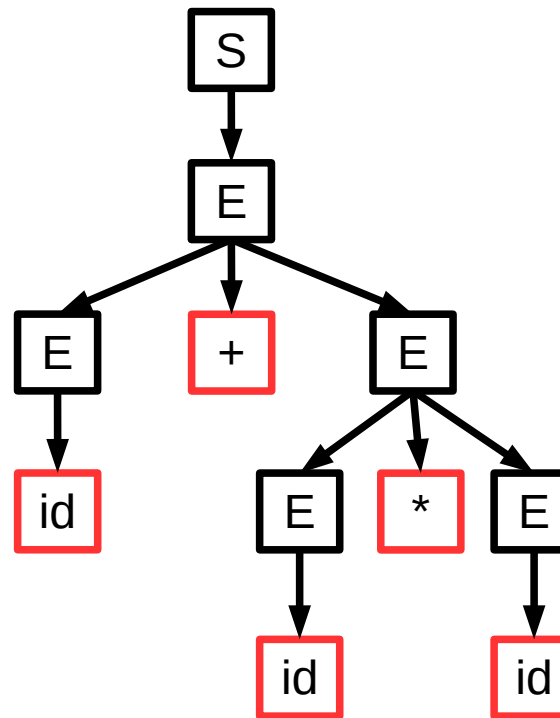
- Ako u svakom rečeničnom obliku izvršimo zamjenu prvog neterminala slijeva dobijamo naredni korak u derivaciji, tako da na kraju dobijemo sljedeću derivaciju i stablo parsiranja:

Stablo parsiranja:

Gramatika:

$S \rightarrow E$
 $E \rightarrow E + E$
 $E * E$
 (E)
 id

$S \rightarrow E$
 $\rightarrow E + E$
 $\rightarrow id + E$
 $\rightarrow id + E * E$
 $\rightarrow id + id * E$
 $\rightarrow id + id * id$



***Lijeva
derivacija
(left-most
derivation)***

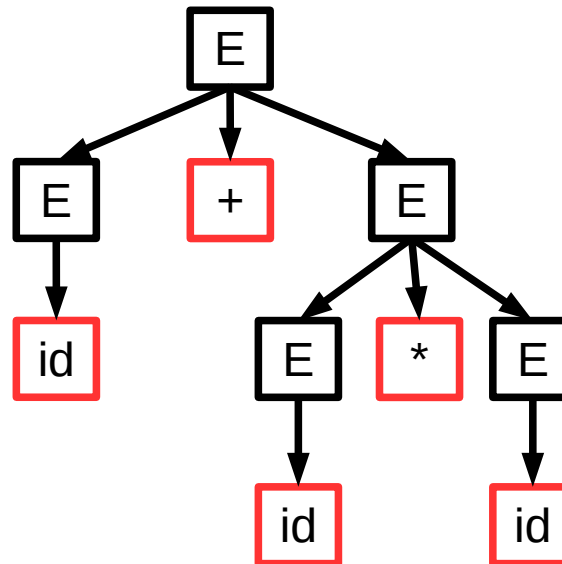
Primjer 2 – desna derivacija

- Ako derivaciju izvedemo tako da u svakom koraku produkcijsko pravilo primjenjujemo na prvi neterminalni simbol sdesna:

Derivacija:

$S \rightarrow E$
 $\rightarrow E + E$
 $\rightarrow E + E * E$
 $\rightarrow E + E * id$
 $\rightarrow E + id * id$
 $\rightarrow id + id * id$

Stablo parsiranja:



Gramatika:
 $S \rightarrow E$
 $E \rightarrow E + E$
 $\quad \quad \quad E * E$
 $\quad \quad \quad (E)$
 $\quad \quad \quad id$

- Stablo parsiranja za lijevu i desnu derivaciju je identično.

***Desna
derivacija
(right-most
derivation)***

Višeznačnost gramatike

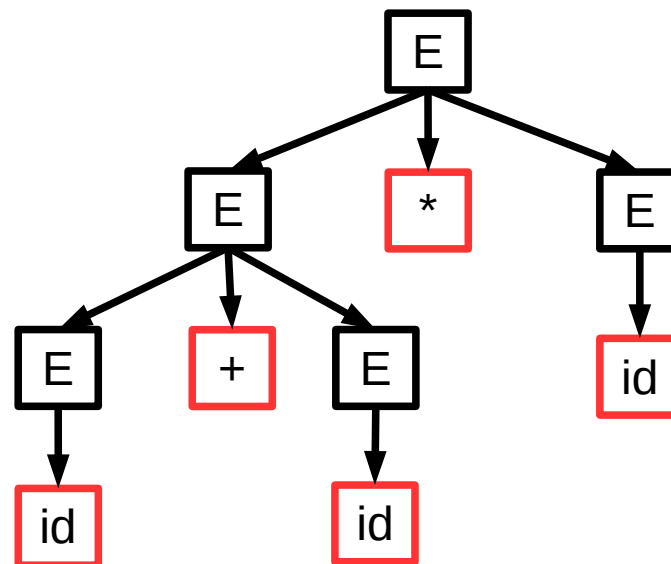
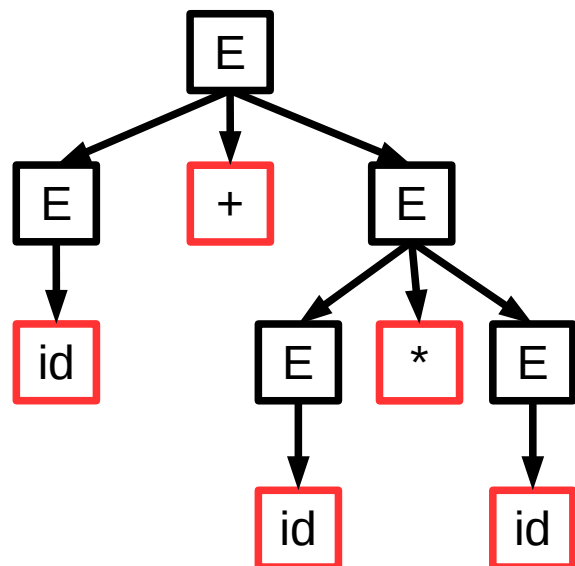
- Za gramatiku koja proizvodi više od jednog stabla parsiranja za istu rečenicu kažemo da je **višeznačna**.

Primjer:

E
 $\rightarrow E + E$
 $\rightarrow id + E$
 $\rightarrow id + E * E$
 $\rightarrow id + id * E$
 $\rightarrow id + id * id$

$E \rightarrow E + E$
 $| E * E$
 $| (E)$
 $| id$

E
 $\rightarrow E * E$
 $\rightarrow E * id$
 $\rightarrow E + E * id$
 $\rightarrow E + id * id$
 $\rightarrow id + id * id$



Višeznačnost

- Višeznačnost gramatike nije dobra i treba je spriječiti. Kako?
 - Napisati gramatiku tako da se spriječi višeznačnost
 - Uvesti "prednost" jednog operatora (tokena) u odnosu na drugi nekom vrstom deklaracije
 - prednost $*$ u odnosu na $+$, putem tzv. asocijativnosti

Sprečavanje višeznačnosti - primjer gramatike

- Za gramatiku:

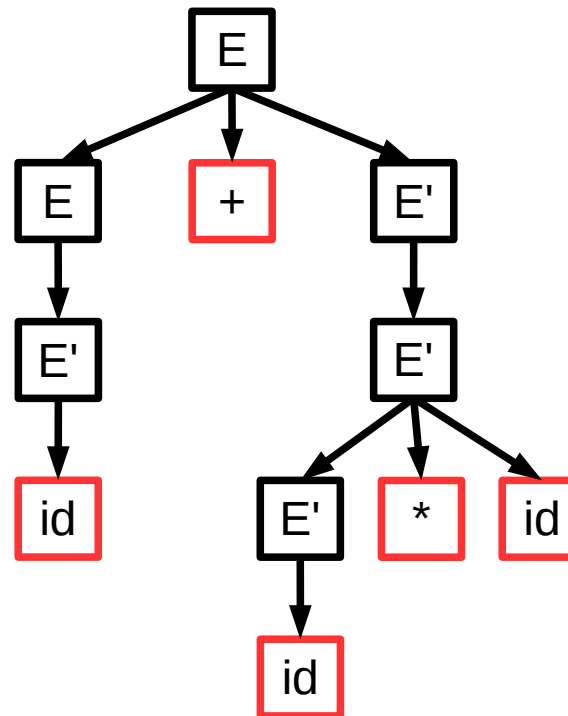
$$E \rightarrow E + E' \mid E'$$
$$E' \rightarrow E' * id \mid id \mid E' * (E) \mid (E)$$

- I ulaznu sekvencu tokena:

id + id * id

Imaćemo:

E
 $\rightarrow E + E'$
 $\rightarrow E' + E'$
 $\rightarrow id + E'$
 $\rightarrow id + E' * id$
 $\rightarrow id + id * i$



Kao što vidimo, E je zadužen za $+$ a E' za $*$.

Sprečavanje višeznačnosti - asocijativnost

- Asocijativnost u programskim jezicima definiše način grupisanja operanada nekog operatora u izrazu bez zagrada.

- Primjer:

$$a - b + c$$

- + i - su lijevo asocijativni pa je gornji izraz ekvivalentan izrazu:

$$(a - b) + c$$

- Kad bi bili desno asocijativni, gornji izraz bi bio ekvivalentan izrazu:

$$a - (b + c)$$

- Koja je asocijativnost operatora * / = u C/C++?

- Asocijativnost se u alatima za generisanje parsera deklariše na poseban način.

Upravljanje greškama (error handling)

- Kompajler treba da:
 - Prevede ispravne programe u ciljni kod.
 - Otkrije neispravne programe i reaguje na odgovarajući način.
- Neke vrste grešaka (npr. u C jeziku):

Vrsta	Primjer	Ko detektuje
Leksička	... int \$a...	Lexer
Sintaksna	... x +/ ...	Parser
Semantička	... int x; x="abc"...	Provjera tipova
Ispravnost programa	Program ispravno preveden	Korisnik / Testiranje programa

*

Upravljanje greškama

- Dobro upravljanje greškama pri kompajliranju treba da uključuje:
 - Precizno i jasno izvještavanje o greškama.
 - Brzi oporavak od greške i nastavak kompajliranja.
 - Ne smije usporavati kompajliranje ispravnog koda.
- Strategije:
 - "Panic mode"
 - Produkcije sa "greškama"
 - Automatska korekcija

“Panic mode”

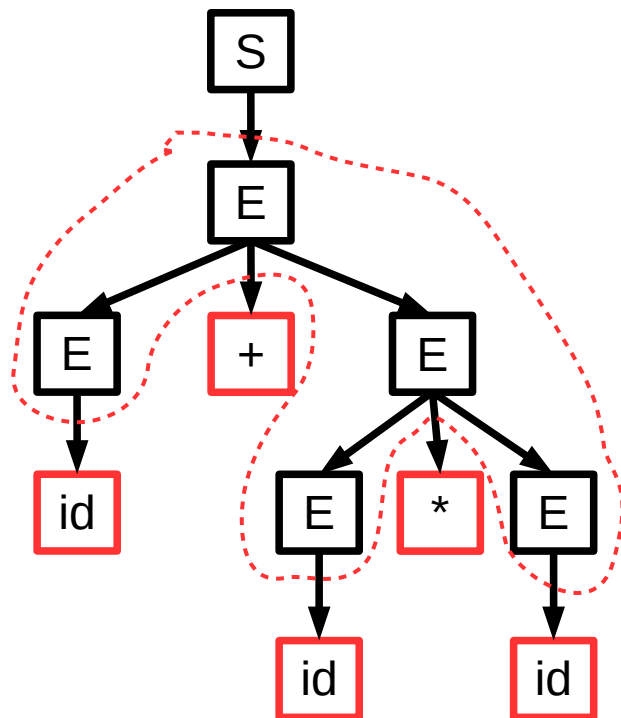
- Najjednostavnija i najpopularnija metoda upravljanja greškama.
- Kad se otkrije greška:
 1. Odbacuju se svi tokeni do onog sa jasnom ulogom.
 2. Nastavi se od tokena iz prethodnog koraka.
- Tzv. "sinhronizirajući" tokeni su obično terminatori izjava:
 - Kraj linije (rečenice)
 - Zatvorena zagrada
 - itd.

Produkcije sa “greškama”

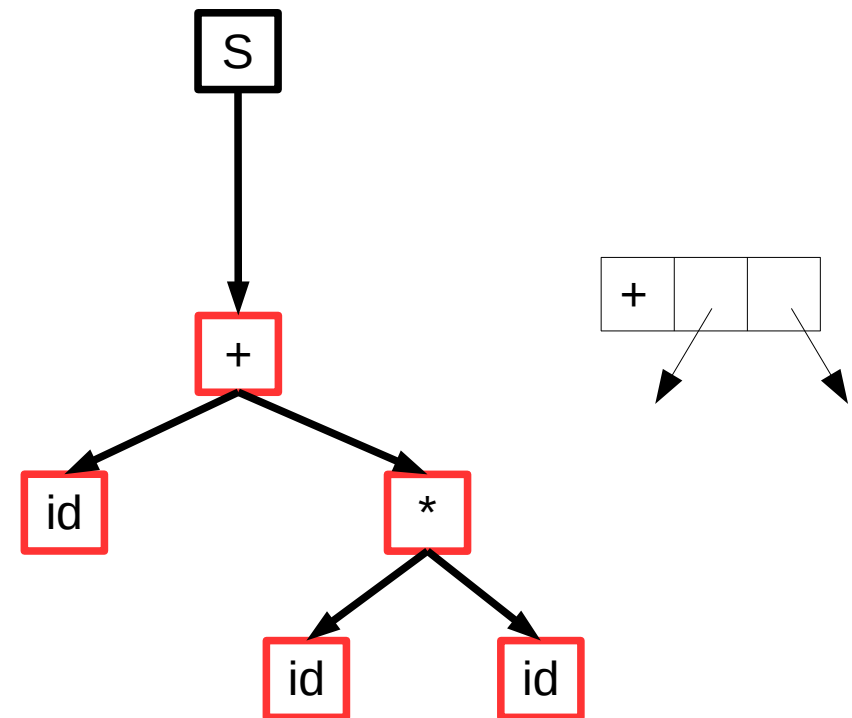
- Dodavanje očekivanih čestih grešaka u gramatiku jezika.
- Primjer:
 - Za prihvatanje izraza **5 x** kao ispravnog sa značenjem **5 * x**
 - Dodati produkciju **E → ... | E E**
- Nedostaci ovog pristupa:
 - Gramatika se dodatno komplikuje.
 - Stimuliše se pisanje “pogrešnog” koda.

Apstraktno sintaksno stablo

- Apstraktno sintaksno stablo – abstract syntax tree (AST).
- Struktura u kojoj, za neki izraz, svaki unutrašnji čvor predstavlja operator a djeca su operandi operacije.
- Razlikuje se od stabla parsiranja u tome što se u AST nalaze samo oni čvorovi koji su zaista potrebni za evaluaciju izraza.



Stablo parsiranja



Apstraktno sintaksno stablo